Genetic Corpus Poisoning for Neural IR

Francesco Busolin[1,*], Claudio Lucchese[1] and Salvatore Orlando[1]

[1]Ca' Foscari University of Venice, Venice, Italy

Abstract

Modern information retrieval (IR) commonly relies on two-stage retrieve-and-rerank pipelines powered by neural Pre-trained Language Models (PLMs). However, these systems are vulnerable to various adversarial attacks, such as corpus poisoning, where a malicious actor injects a small number of adversarial passages to manipulate search results. While prior attacks often rely on white-box gradient information or computationally expensive surrogate models, we propose Genetic Corpus Poisoning (GCP), a novel, fully gradient-free method for generating adversarial documents. Specifically, we propose a query-centered genetic approach to generate documents that are collected by a dense retrieval model and are then highly ranked by a cross-encoder. Experiments on popular benchmark datasets and models show that this approach is highly effective against various retriever-ranker pairs, achieving attack success rates exceeding 90% and a 64% degradation in system effectiveness in terms of nDCG@10.

Keywords

Adersarial ML, Neural Information Retrieval, Retrieve and Re-rank, Genetic algorithms

1. Introduction

Pre-trained Language Models (PLMs) based on transformer networks achieve state-of-the-art performance in ad-hoc search. However, their high computational cost makes balancing efficiency with ranking quality an ongoing challenge [1, 2]. A common strategy to reduce overall computational requirements is to adopt a *retrieve-and-re-rank* pipeline, in which accurate PLMs are applied only at the re-ranking stage, after a cheaper first-stage retrieval has narrowed the candidate set [3, 4]. This two-stage design, illustrated in Figure 1, allows the system to preserve ranking quality while keeping inference costs manageable.

Despite their effectiveness, two-stage ranking pipelines are inherently vulnerable to adversarial manipulation [5]. Typical first-stage retrievers rely on a *dense retriever* based on sentence transformers, which maps queries and documents into a shared embedding space and ranks candidates by vector similarity [6]. This allows the precomputation of all the sentence-level embeddings of the documents in the corpus and their organization into efficient indexes [7, 8]. While highly effective for relevance matching, this embedding-based ranking can be exploited by crafting documents that are artificially close to target queries in the embedding space, causing them to be retrieved despite being irrelevant or malicious [9, 10, 11]. The second stage, a *cross-encoder reranker*, performs a more fine-grained pairwise relevance assessment and is generally more accurate. Nevertheless, even cross-encoders have been shown to suffer from similar vulnerabilities, in which a malicious user can feed them crafted samples that receive high scores [12, 13].

In this work, we investigate the problem of *corpus poisoning* in neural information retrieval, in which a malicious actor injects a small number of adversarial passages into a document corpus to induce the system to retrieve them in response to user queries. This threat model is particularly concerning in practice, as it requires no modification to the retrieval system itself: the attacker operates solely by manipulating the corpus, which is often partially open to external contributions.

Existing work has demonstrated a wide range of remarkably effective attacks against neural IR systems. However, nearly all of them rely on gradient information to guide the generation of adversarial

IIR 2026: 16th Italian Information Retrieval Workshop, November 12–14, 2026, Rome, Italy

*Corresponding author.

francesco.busolin@unive.it (F. Busolin); claudio.lucchese@unive.it (C. Lucchese); orlando@unive.it (S. Orlando)

0000-0002-3235-2524 (F. Busolin); 0000-0002-2545-0425 (C. Lucchese); 0000-0002-4155-9797 (S. Orlando)

© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

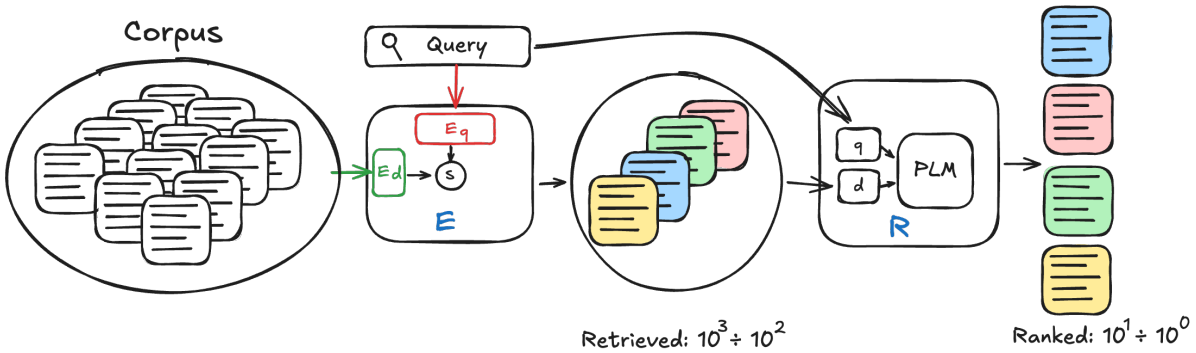


Figure 1: A typical retrieve and re-rank pipeline

passages. A first line of work operates in a *white-box* setting, where the attacker has full access to the target neural model and can directly compute its gradients to craft adversarial inputs [5, 14, 13, 15]. A second, more practically motivated line of work assumes a *black-box* setting, but circumvents the lack of direct access by employing Pseudo-Relevance Feedback (PRF) against the target model to collect soft relevance signals, which are then used to train a surrogate model whose gradients drive the attack [10, 16, 17, 18]. While the latter setting is more realistic, it still assumes that the attacker can construct a differentiable approximation of the target system's behavior.

While crafting attacks using gradients or gradient-based proxies is, in principle, a viable and effective approach, we argue that targeting the entire pipeline makes gradient computation significantly more challenging and less straightforward. In particular, the lack of a direct connection between the different components makes it unclear how gradients should be propagated across models during backpropagation. Moreover, from a practical perspective, developing, training, and deploying surrogate models can be both computationally expensive and time-consuming.

We depart from these assumptions and propose Genetic Corpus Poisoning (GCP), a novel, fully *gradient-free* method for generating adversarial passages. Our approach is based on a genetic algorithm that relies exclusively on the output scores of the target models, treated as black-box scoring functions, to guide the search for effective adversarial documents.

Furthermore, unlike most prior work that targets only the retrieval stage, we demonstrate that our approach is effective against the *full retrieve-and-rerank pipeline*. Specifically, our method jointly optimizes adversarial documents to achieve high scores from both the retriever and the re-ranker, pushing the document embedding close to the query representation in the retriever's embedding space while simultaneously obtaining a high relevance probability from the cross-encoder re-ranker. The ability to fool both stages with a single document exposes a deeper vulnerability in the scoring mechanisms of these models, suggesting that the two stages share exploitable weaknesses despite their architectural differences.

To this end, it is important to note that in this work, we are not interested in crafting subtly modified samples that retain their semantic content almost intact; instead, we want to fully expose the vulnerability of PLMs in placing documents in top-ranking positions, even when they contain pure gibberish.

Tests conducted on a diverse array of both retrievers and re-ranking models show that we can generate adversarial documents that reliably fool dense-ranking pipelines in more than 90% of cases. Moreover, we show that we can place such documents in the very top positions, mining almost entirely the effectiveness of the models tested, with reductions in nDCG@10 of up to 64%.

Lastly, we show an example of an injection attack using GCP, in which we embed an arbitrary text payload in the adversarial samples while still being able to place them in top positions in the rankings.

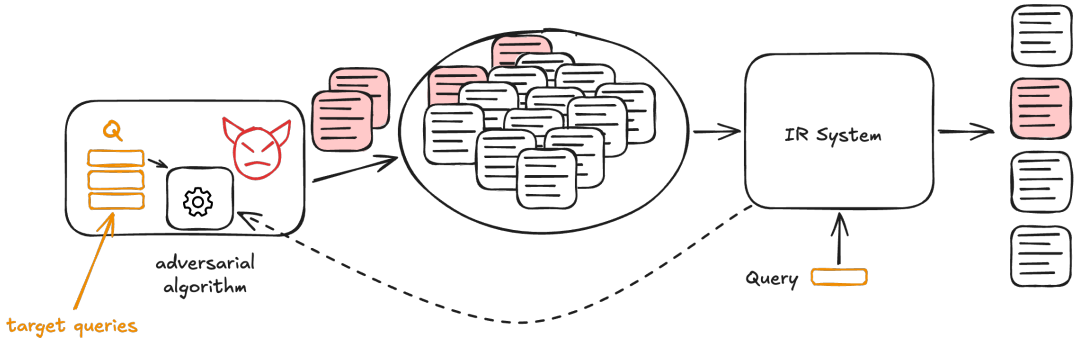


Figure 2: General schema of a Targeted Corpus Poisoning Attack. The attacker generates adversarial samples from a set of target queries and by probing the target IR system.

2. Related Work

In Adversarial Information Retrieval, the primary case of study for manipulation is targeted document promotion [19, 20]: artificially inflating the rank of a specific document, typically a low-quality or irrelevant page boosted to erode search quality and the user experience [21, 10], or one an attacker promotes unethically, a form of black-hat Search Engine Optimization (SEO) [22]. Much research has consequently addressed adversarial ranking attacks [23, 21, 10, 9], which alter a document imperceptibly, yet misleading for the retrieval model, causing its ranking to improve for one or more target queries [10, 24, 16]. A related but more disruptive vector forgoes stealthy promotion and instead degrades the reliability and trustworthiness of the system as a whole, exposing users to non-relevant, incoherent, or outright nonsensical content [17, 11].

We instead take the perspective of the queries, mounting a *query-centric* corpus poisoning attack, in which the adversary either disrupts the system's effectiveness on a set of predefined target queries or uses them as triggers to retrieve the injected documents at a later time.

Among recent corpus poisoning proposals, we compare against Zhong et al. [9], hereafter HotFlip, which generates one passage p at a time starting from a random corpus passage. At each step it samples a token t_i and scores its replacement by any vocabulary token t' with the HotFlip [25] approximation

$$e_{t_i'}^{\top} \nabla_{e_{t_i}} \mathrm{sim}(q, p)$$

where $\nabla_{e_{t_i}} \mathrm{sim}(q, p)$ is the gradient with respect to the token embedding e_{t_i}, keeping the token that maximizes it over a set of training queries. For multiple documents it clusters those queries with k-means and generates one per cluster, accumulating gradients only on that cluster's queries.

3. Methodology

We tackle the standard *retrieve-and-rerank* paradigm, in which a retriever first scans the entire document corpus to retrieve a subset of K candidates (e.g., $K = 100$), which are then re-ranked by a more accurate model to produce the final ranking. Performance is typically evaluated on the top-k documents, with $k \ll K$ (e.g., $k = 10$). Unlike prior work, we propose to jointly attack both stages of the pipeline by crafting adversarial passages that can evade both the retriever and the re-ranker simultaneously.

In this work, we use single-vector dense models as retrievers and cross-encoders as re-rankers [26, 27, 28, 29]. Specifically, the retrievers follow a dual-encoder architecture, encoding queries and documents independently via a query encoder $Enc_q(\cdot)$ and a document encoder $Enc_d(\cdot)$. Rather than using models optimized for dot-product similarity, we employ retrievers trained with cosine similarity as the relevance score:

$$\mathrm{E}(q, d) = \cos(q, d) = \frac{Enc_q(q)^{\top} Enc_d(d)}{\|Enc_q(q)\| \cdot \|Enc_d(d)\|}. \tag{1}$$

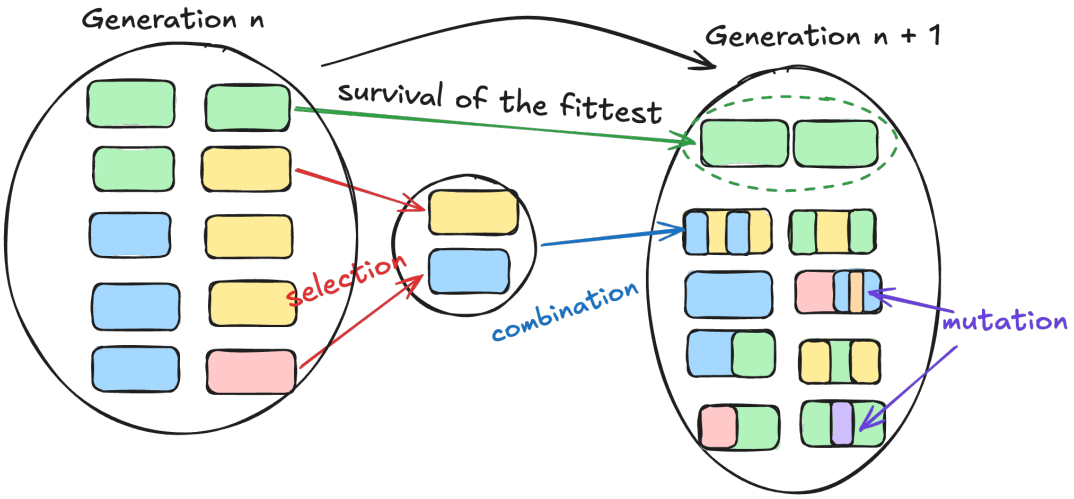


Figure 3: Schema of a generic Genetic Algorithm

The re-rankers, by contrast, are cross-encoders that jointly process the query and document by taking their concatenation as input to a single dense network and output a single score $R(q, d)$ modeling the probability of d being relevant for q.

3.1. Threat Model

We frame GCP as a *Targeted Corpus Poisoning* attack, in which a dedicated set of adversarial documents is crafted for each target query. Formally, given a query q_i, our goal is to generate a set of adversarial documents $\mathscr{C}_i^- = \{d_{i1}^-, d_{i2}^-, \ldots, d_{ij}^-\}$ that, when injected into the corpus to form $\mathscr{C}^* = \mathscr{C} \cup \mathscr{C}_i^-$, cause at least one adversarial document to appear among the final top-k results for q_i. Since we target the full retrieve-and-rerank pipeline, this requires each adversarial document to clear both stages: it must first be retrieved among the top-K candidates by $E(\cdot,\cdot)$ *and* then be ranked among the top-k results by $R(\cdot,\cdot)$. Formally, we want some $d^- \in \mathscr{C}_i^*$ such that:

$$E(q_i, d^-) \geq E(q_i, d^{(K)}), \qquad R(q_i, d^-) \geq R(q_i, d^{(k)}), \tag{2}$$

where $d^{(K)}$ and $d^{(k)}$ denote the K-th and k-th ranked documents in $\mathscr{C}^*$ under $E(\cdot)$ and $R(\cdot)$, respectively.

An attack is considered successful if at least one adversarial document appears in the final top-k results presented to the user. It is worth noting that imperceptibility is not a requirement of our attack [30]. Rather, we consider an attack successful when a user encounters an adversarial document in the top results and perceives it as irrelevant or unhelpful, despite the system having ranked it highly.

To assess the effectiveness of our attack, we consider three evaluation metrics. The primary metric is the *Success Rate* (SR@k), defined as the ratio of queries for which at least one adversarial document d^- appears in the top-k results. We also report the *Saturation* (SAT@k), which measures the number of adversarial documents in the top-k, capturing how thoroughly the ranking is contaminated. Naturally, SAT@k is strongly influenced by the number of adversarial documents injected into the corpus for each query q_i, namely $|\mathscr{C}_i^-|$. Finally, we report nDCG@k to quantify the overall degradation in ranking quality induced by the adversarial documents. Indeed, a secondary objective of GCP is to generate a sufficient number of adversarial documents to compromise a large portion of the top-k. In other words, we want not only to place some adversarial samples in the top-k but also, ideally, to have all of the top-k be adversarial samples.

3.2. Genetic Adversarial Attack

We generate our adversarial samples with a genetic algorithm because our attack targets the IR pipeline as a whole, which in our setting comprises different models in sequence. A gradient-based attack

would have to backpropagate through all of them, becoming a non-trivial and potentially infeasible endeavor when non-differentiable operations such as sorting sit between stages, or when the models are unknown or inaccessible. A genetic algorithm sidesteps this entirely by treating the pipeline as a chain of black-box scoring functions and driving the evolutionary process directly from their outputs.

A genetic algorithm (GA) is a heuristic search and optimization technique inspired by natural selection (Figure 3): by mimicking biological evolution, it repeatedly modifies a population of candidate solutions to tackle problems poorly suited to traditional algorithms. It rests on three operators:

- *Selection*: scores every candidate in the current population through a fitness function and samples parents at random, weighted by those fitness values.
- *Recombination (crossover)*: combines the genetic traits of the selected parents into one or more distinct offspring for the next generation.
- *Mutation*: applies slight random alterations to the offspring, maintaining genetic diversity and preventing the algorithm from getting "stuck" on suboptimal solutions.

Fitness Function The fitness function is a key component, as it determines the quality of each candidate and hence its likelihood of being selected for reproduction. Since our goal is to produce documents that score highly under *both* the retrieval model $E(\cdot)$ and the re-ranker $R(\cdot)$, it must reflect joint performance across the two stages; we therefore define it as the harmonic mean of the two scores:

$$\mathscr{F}(q,d) = \frac{2}{\frac{1}{E(q,d)} + \frac{1}{R(q,d)}} \tag{3}$$

with $E(q,d), R(q,d) \in [0,1]$, where 0 denotes a completely ineffective document and 1 a perfect one. Dual-encoder scores are cosine similarities lying in $[-1,1]$, which we rescale linearly as

$$\mathrm{E}(q,d) = \frac{1}{2}(\cos(q,d) + 1)\,, \tag{4}$$

whereas the cross-encoders we select already output a relevance probability, naturally in $[0,1]$.

We prefer the harmonic over the arithmetic mean because it penalizes imbalanced scores more severely. A document with $R(q,d) = 0.95$ and $E(q,d) = 0.05$ would never be retrieved in practice, yet the arithmetic mean rates it at $\mathscr{F}(q,d) = 0.5$, implying moderate quality; the harmonic mean assigns $\mathscr{F}(q,d) \approx 0.095$, correctly reflecting that such a document is useless for our purposes.

Generation Process To generate adversarial documents, we adopt an evolutionary strategy. For each target query q, we maintain a population of N candidate documents. Each document is initialized by sampling tokens uniformly at random from the target model's vocabulary, with its length drawn uniformly from a predefined range $[L_{\min}, L_{\max}]$. The population is then evolved for M generations according to the fitness function, with each generation comprising the following steps.

1. **Carrying Through**: The top 10% of documents, ranked by their fitness value, are carried over to the next generation unchanged.
2. **Recombination**: Two parent documents A and B are sampled at random from the current population, with probability proportional to their fitness. To produce a child document, both parents are first padded to the same length using the `[PAD]` token; each position i of the child is then filled with either $A[i]$ or $B[i]$, chosen with equal probability. All padding tokens are subsequently removed. This process is repeated until the new population reaches the same size as the previous one.
3. **Mutation**: After recombination, each token in the newly generated document, with probability equal to a mutation rate μ, is independently replaced by a token sampled uniformly from the vocabulary. This mutation step introduces tokens that may not have been present in the current population.

4. **Migration**: As an additional step, each document undergoes a migration step. Here, each token in the document has a $\mu/2$ probability of swapping positions with another token in the same document. This allows the algorithm to reorder tokens without relying solely on mutation. While migration can, in principle, be replicated by pairs of mutations where two tokens mutate into each other, preliminary experiments showed that including it as an explicit step accelerates convergence.

Performing the Attack After the M iterations, we obtain a collection of N token sequences representing our adversarial samples. To execute GCP, we select the β highest-scoring samples based on their final fitness scores. These β documents constitute our $\mathscr{C}_i^-$, i.e., $\beta = |\mathscr{C}_i^-|$, and are injected into $\mathscr{C}$ to form the poisoned corpus $\mathscr{C}^*$.

Attack Cost To conduct GCP, we have to run through the pipeline $N \cdot M$ times, since for each of the N samples, we need to compute the fitness function M times. However, since we do not need gradient information and do not require training, these computations are feasible even on limited hardware, where, in the extreme case, only one sample is scored at a time. From our experiments, we found that we need about 5 minutes of GPU time on an NVIDIA A100 to execute one attack with $N = 100$ and $M = 2000$, which is much lower than HotFlip, which takes approximately 48 hours to generate 10 adversarial samples for MS-MARCO on the same hardware.

4. Experimental Evaluation

4.1. Baselines

To assess the effectiveness of our proposed method, we compare it against both heuristics and gradient-based methods. In particular, we compare with the intuitive approach of prepending the target query to β random documents in the corpus and treating them as adversarial samples; we call this attack PrepQ. Similarly, we try to randomly place the query tokens inside the same sampled documents; we call this attack RandQ. Lastly, we compare against HotFlip [9] as our primary benchmark, which, as a reminder, is a white-box attack for corpus poisoning. HotFlip was first presented to attack dense sentence-level retrievers like Contriever [31], DPR [32], and ANCE [33].

In addition to these attacks, we also test a variant of GCP in which, after the generation, the query tokens are added at the beginning of the generated documents; we call this attack GCP $_{\text{PrepQ}}$, as it effectively is a union between our method and the PrepQ baseline.

4.2. Experimental Setup

All experiments run on a single NVIDIA A100 GPU.

GCP starts from $N = 100$ randomly initialized sequences of $L_{\min} = 64$ to $L_{\max} = 128$ tokens drawn from the vocabulary, and runs the genetic algorithm for $M = 2000$ steps with a mutation rate decaying from $\mu_0 = 0.2$ as

$$\mu_{i+1} = \frac{1}{2}\mu_0 + \frac{1}{2} \cdot \frac{\mu_0}{1 + \log(i+1)} \tag{5}$$

For HotFlip we reuse the authors' code and default settings[1], running the optimization for 5000 iterations; its prohibitive cost limits us to 10 generated documents, which also caps β in all experiments. Remaining hyperparameters follow the original paper [9].

Attacks are applied per query: we inject the adversarial samples into the index, evaluate the query, and restore the index before moving on. HotFlip documents are generated once for MS-MARCO, and the first β of them are injected unchanged for every query, simulating a one-off poisoning; PrepQ and RandQ operate on β documents sampled uniformly at random from the index; GCP injects the β documents with the highest fitness for the query at hand.

[1] https://github.com/princeton-nlp/corpus-poisoning

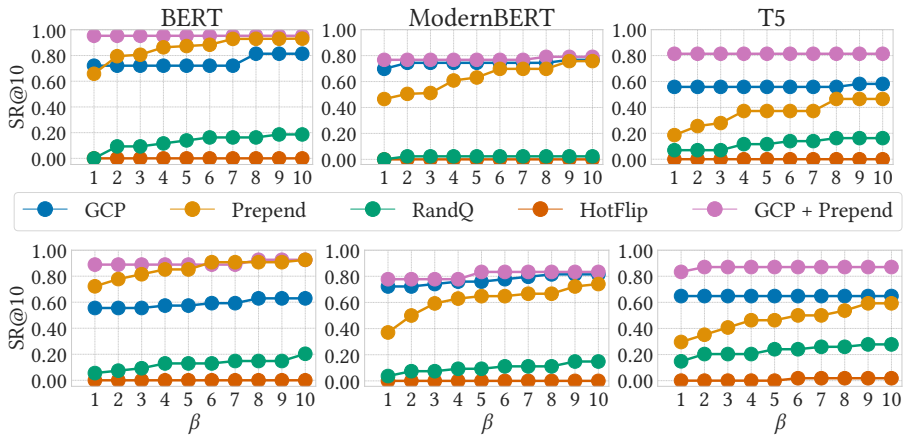


Figure 4: Success Rate on TREC-DL 2019 (top) and TREC-DL 2020 (bottom)

Datasets. We experiment on MS-MARCO Passage [34], a corpus of roughly 8.8 million short web passages built by retrieving Bing candidates for a large pool of real user queries and annotating them for relevance. Target queries come from the TREC Deep Learning passage ranking sets: TREC-DL 2019 [35] and 2020 [36], comprising 43 and 54 fully judged queries, respectively.

Models. We evaluate three backbone families:BERT [37], ModernBERT [38] and T5 [39], all available on HuggingFace [40]. Pairing for each a dual-encoder retriever with a cross-encoder re-ranker, both fine-tuned on MS MARCO. These are SentenceBERT [6][2] with MonoBERT [41][3]; SentenceModernBERT[4] with Reranker-ModernBERT[5]; and SentenceT5 [43][6] with RankT5 [44][7]. The two BERT-based models have 12 transformer blocks each, totalling 109M parameters for MonoBERT and 66M for SentenceBERT, whose backbone is DistilBERT; the two ModernBERT-based models share an architecture of 22 blocks and 149M parameters. RankT5 follows the full T5 encoder–decoder design with 12 blocks per stack and 223M parameters, whereas SentenceT5 keeps only the 12-block encoder, for 110M.

Each family uses its own tokenizer: WordPiece [45] with 30,522 tokens for BERT, BPE [46] with 50,368 for ModernBERT and SentencePiece [47] with 32,128 for T5. For GCP we further restrict these to English-only tokens, excluding special and CJK ones, leaving vocabularies of 27,621 (BERT), 47,015 (ModernBERT) and 29,557 (T5).

4.3. Attack Performance

We now test GCP, HotFlip, and the two heuristics PrepQ and RandQ on the full retriever-re-ranker pipeline.

Success Rate. Figure 4 plots success rate against the number of adversarial documents injected per query. On BERT, PrepQ and GCP $_{\text{PrepQ}}$ dominate, exceeding 80% and 90% respectively on both TREC-DL datasets: at $\beta = 10$, PrepQ reaches an SR@10 of 0.931 (TREC-DL 2019) and 0.926 (TREC-DL 2020), and GCP $_{\text{PrepQ}}$ 0.953 and 0.926 (Table 1, Table 2). Merely prepending the query text to a document is therefore remarkably effective even in a multi-stage setting. GCP ranks second at $\beta = 1$ on TREC-DL 2019, whereas RandQ peaks at 20% and HotFlip never places a single adversarial sample in MonoBERT's top-10, regardless of budget. Even under the laxer SR@100, which measures the fraction of queries retrieving at least one adversarial sample from the first stage, HotFlip reaches at most 70%, against more than 90% for all other attacks and 100% for PrepQ.

[2] `sentence-transformers/msmarco-distilbert-cos-v5`
[3] `veneres/monobert-msmarco` [42]
[4] `joe32140/ModernBERT-base-msmarco`
[5] `tomaarsen/reranker-modernbert-base-msmarco-bce`
[6] `sentence-transformers/sentence-t5-base`
[7] `Soyoung97/RankT5-base`

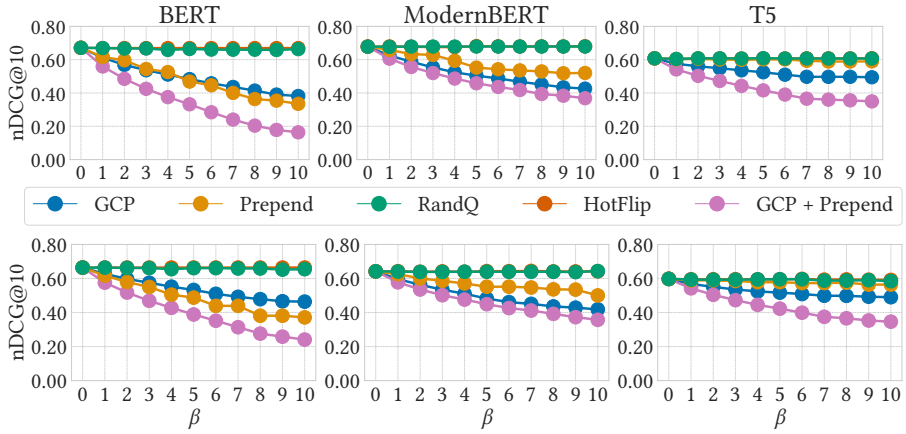


Figure 5: Impact on nDCG@10 on TREC-DL 2019 (top) and TREC-DL 2020 (bottom)

		BERT				ModernBERT				T5			
Attack	β	SR@10	SR@100	SAT@10	nDCG@10	SR@10	SR@100	SAT@10	nDCG@10	SR@10	SR@100	SAT@10	nDCG@10
No Attack	0	–	–	–	0.671	–	–	–	0.678	–	–	–	0.608
RandQ	1	0.000	0.930	0.00	0.670	0.000	1.000	0.00	0.678	0.070	0.953	0.07	0.604
RandQ	5	0.140	0.977	0.21	0.664	0.023	1.000	0.02	0.678	0.116	0.953	0.23	0.604
RandQ	10	0.186	0.977	0.30	0.663	0.023	1.000	0.02	0.678	0.163	1.000	0.35	0.604
PrepQ	1	0.658	1.000	0.66	0.616	0.465	0.977	0.47	0.658	0.186	0.953	0.19	0.602
PrepQ	5	0.874	1.000	3.26	0.469	0.631	1.000	1.87	0.552	0.372	0.953	0.88	0.602
PrepQ	10	0.931	1.000	*5.70*	0.335	0.757	1.000	2.70	0.520	0.465	1.000	1.23	0.590
HotFlip	1	0.000	0.419	0.00	0.670	0.000	1.000	0.00	0.678	0.000	0.884	0.00	0.604
HotFlip	5	0.000	0.628	0.00	0.670	0.000	1.000	0.00	0.678	0.000	0.907	0.00	0.604
HotFlip	10	0.000	0.674	0.00	0.670	0.000	1.000	0.00	0.678	0.000	0.907	0.00	0.604
GCP	1	0.721	1.000	0.72	0.611	0.698	1.000	0.70	0.625	0.558	0.953	0.56	0.575
GCP	5	0.721	1.000	3.21	0.483	0.744	1.000	2.84	0.504	0.558	0.953	2.58	0.524
GCP	10	0.814	1.000	4.91	0.382	0.767	1.000	*4.07*	0.426	0.581	0.953	*3.58*	0.494
GCP $_{\text{PrepQ}}$	1	**0.953**	1.000	0.95	0.559	**0.767**	1.000	0.77	0.606	**0.814**	0.977	0.81	0.541
GCP $_{\text{PrepQ}}$	5	0.953	1.000	4.65	0.332	0.767	1.000	3.53	0.458	0.814	0.977	3.88	0.416
GCP $_{\text{PrepQ}}$	10	0.953	1.000	***7.86***	0.164	0.791	1.000	***5.98***	0.369	0.814	0.977	***6.02***	0.349

Table 1
TREC DL 2019. For $\beta = 1$, we report the most successful attack, in terms of SR@10, in **bold** and underline the second best. We do the same for SAT by *marking* the **best** and second best values with $\beta = 10$

On ModernBERT, GCP $_{\text{PrepQ}}$ is the strongest attack and GCP a close second, both around 80% on either dataset, while PrepQ becomes comparable to GCP as β grows and RandQ and HotFlip again contribute little, with HotFlip once more failing to reach the top-10. Unlike BERT, every method here retrieves at least one adversarial sample for every query, attaining SR@100 = 1.0 already at $\beta = 1$, except PrepQ, which needs at most 5 injected documents. T5 is the most robust pipeline, capping GCP just below 60%, some 12 points above PrepQ; yet combining the two in GCP $_{\text{PrepQ}}$ pushes success rates past 80% again, a first indication that the two strategies synergize. HotFlip and RandQ remain ineffective here as well.

Quality Impact. Figure 5 reports the impact of each attack on nDCG@10. We exclude HotFlip and RandQ from the discussion: although both place adversarial samples in the top-k neither displaces any relevant result, leaving nDCG@10 essentially unchanged.

On BERT, PrepQ attacks more queries than GCP, yet the two degrade retrieval quality to a similar extent: GCP compensates with severity, ranking its adversarial documents higher. As β grows, however, the success rate of PrepQ outweighs this effect and it becomes the strongest single attack. SAT@10

		BERT				ModernBERT				T5			
Attack	β	SR@10	SR@100	SAT@10	nDCG@10	SR@10	SR@100	SAT@10	nDCG@10	SR@10	SR@100	SAT@10	nDCG@10
No Attack	0	–	–	–	0.664	–	–	–	0.642	–	–	–	0.598
RandQ	1	0.056	0.963	0.06	0.664	0.037	1.000	0.04	0.642	0.148	0.907	0.15	0.595
RandQ	5	0.130	0.981	0.24	0.661	0.093	1.000	0.11	0.641	0.241	1.000	0.70	0.595
RandQ	10	0.204	0.981	0.41	0.655	0.148	1.000	0.24	0.641	0.278	1.000	1.13	0.584
PrepQ	1	0.722	1.000	0.72	0.617	0.370	0.963	0.37	0.625	0.296	0.944	0.30	0.590
PrepQ	5	0.852	1.000	3.09	0.488	0.630	1.000	1.54	0.571	0.463	1.000	1.26	0.578
PrepQ	10	0.926	1.000	*5.39*	0.372	0.722	1.000	2.57	0.535	0.537	1.000	1.87	0.576
HotFlip	1	0.000	0.463	0.00	0.664	0.000	1.000	0.00	0.642	0.000	0.889	0.00	0.595
HotFlip	5	0.000	0.667	0.00	0.664	0.000	1.000	0.00	0.642	0.000	0.963	0.00	0.595
HotFlip	10	0.000	0.704	0.00	0.664	0.000	1.000	0.00	0.642	0.019	0.963	0.02	0.592
GCP	1	0.556	0.963	0.56	0.627	0.722	1.000	0.72	0.596	0.648	0.907	0.65	0.568
GCP	5	0.574	0.963	2.43	0.533	0.759	1.000	3.24	0.482	0.648	0.907	3.00	0.517
GCP	10	0.630	0.981	3.78	0.464	0.815	1.000	*4.43*	0.419	0.648	0.907	*4.06*	0.490
GCP $_{\text{PrepQ}}$	1	**0.889**	0.981	0.89	0.576	**0.778**	1.000	0.78	0.578	**0.833**	0.981	0.83	0.542
GCP $_{\text{PrepQ}}$	5	0.889	0.981	4.30	0.389	0.833	1.000	3.61	0.449	0.870	1.000	4.13	0.423
GCP $_{\text{PrepQ}}$	10	0.926	0.981	***7.09***	0.241	0.833	1.000	***5.65***	0.357	0.870	1.000	***6.41***	0.346

Table 2
TREC DL 2020. For $\beta = 1$, we report the most successful attack, in terms of SR@10, in **bold** and underline the second best. We do the same for SAT by *marking* the **best** and second best values with $\beta = 10$

	TREC-DL 2019		TREC-DL 2020	
	1	10	1	10
BERT	$0.674_{-6.53\%}$	$0.744_{-8.60\%}$	$0.444_{-20.14\%}$	$0.481_{-23.65\%}$
ModernBERT	$0.605_{-13.32\%}$	$0.651_{-15.12\%}$	$0.685_{-5.12\%}$	$0.778_{-4.54\%}$
T5	$0.512_{-8.25\%}$	$0.535_{-7.92\%}$	$0.574_{-11.42\%}$	$0.611_{-5.71\%}$

Table 3
Injection Attack using GCP: SR@10 obtained with $\beta = 1$ and $\beta = 10$

(Table 1, Table 2) agrees: GCP places slightly fewer adversarial documents in the top-k on both datasets, except on TREC-DL 2019 with $\beta = 1$. GCP $_{\text{PrepQ}}$ remains the best strategy overall, with 1.4× (TREC-DL 2019) and 1.3× (TREC-DL 2020) more adversarial documents in the top-k than PrepQ.

The ordering reverses on the larger pipelines. On ModernBERT, GCP inserts 1.5× more adversarial samples than PrepQ on TREC-DL 2019, with GCP $_{\text{PrepQ}}$ adding a further 10–46%, and opens a clear gap on TREC-DL 2020 (2.34× higher SAT@10, 3.61 vs. 1.54). On T5, PrepQ barely affects retrieval effectiveness, whereas GCP with $\beta = 10$ costs 11 nDCG@10 points on TREC-DL 2019 and 10 on TREC-DL 2020, against 1 and 2 for PrepQ. Quality reduction also confirms the synergy noted above: on T5 the SAT metrics of GCP $_{\text{PrepQ}}$ exceed the sum of those of the two attacks in isolation, and nearly so on ModernBERT.

In summary, on BERT PrepQ is a simple and highly effective poisoning method, but it loses ground to more sophisticated approaches as pipeline size grows, with GCP consistently superior. Concretely, on TREC-DL 2020 with $\beta = 10$, GCP places 1.72× more adversarial samples in the top-10 than PrepQ on ModernBERT, yielding an nDCG@10 21.68% lower (0.419 vs. 0.535); on T5 it achieves a SAT@10 2.17× higher than PrepQ, for a relative nDCG@10 decrease of 14.9%. Naive text forgeries thus become progressively less effective as model complexity increases.

4.4. Payload Injection Attack

Finally, we consider a case study in which a malicious actor conceals private information within the document index, so that it can later be retrieved by issuing a trigger query to the system, an attack commonly known as *data exfiltration* [48, 49]. Since a direct link between the attacker and the user who exploits the attack is undesirable [49], both PrepQ and GCP $_{\text{PrepQ}}$ are unsuitable here: either would introduce the trigger into the exfiltrated document.

We therefore extend GCP with an additional step during generation, which we term *injection*: with probability equal to the mutation rate μ, a payload message is inserted into the candidate document at a random point. To further steer the search toward payload-bearing documents, we halve the fitness score of any sample that does not contain the payload. We experiment across all pipelines using `key[sec@ret]` as the payload, reporting results for GCP alone in Table 3 to better isolate how well arbitrary text can be embedded in our samples.

Relative to the standard attack formulation, every pipeline loses success rate on both datasets, from modest ($\approx$ 5%) to moderate ($\approx$ 20%) which is expected, given the additional constraint imposed on the generated documents. Even so, a single injected document still attacks at least 44% of queries, indicating that a dedicated adversary could reliably embed sensitive information into the index and retrieve it later.

4.5. Final Remarks

The gap in attack effectiveness between GCP and HotFlip is, to some extent, expected: GCP is explicitly re-ranker-aware and shapes passages as a compromise between the two pipeline stages, whereas HotFlip optimizes at the corpus level against groups of queries rather than specific ones, and remains oblivious to any scoring beyond retrieval. More revealing is that passages engineered to deceive one model can be wholly ineffective against another, even an architecturally similar one, pointing to deeper divergences in how the two stages learn to separate relevant from irrelevant content. The inverse is equally striking: semantically vacuous text can score highly on both retriever and re-ranker, so despite their architectural and objective differences the two stages share exploitable blind spots. Both observations call for a closer look at what patterns these models learn to reward, and at how relevance signals are encoded, propagated and turned into a ranking decision.

Limitations GCP is effective across diverse pipelines, but its adversarial samples are inherently unnatural and therefore relatively straightforward to detect and filter with standard defensive methodologies; being a query-specific attack, its generalizability to queries beyond those directly targeted also remains uncertain. We nonetheless argue that GCP sheds new light on fundamental vulnerabilities in modern IR pipelines, revealing that PLMs can assign high relevance scores to texts that are, from a human perspective, essentially meaningless.

5. Conclusions and Future Work

In this work, we presented GCP, a query-specific, gradient-free adversarial attack against dense IR pipelines for corpus poisoning. We showed that GCP can reliably compromise a diverse array of retrieve-and re-rank pipelines, with efficacy often surpassing 70% on standard benchmarking datasets. We compared GCP with some heuristics and HotFlip, a white-box attack, confirming that despite the lack of gradient information, GCP can craft effective adversarial samples. As future work, we plan to expand the study of GCP applicability to additional scenarios, including multi-representation models such as ColBERT [50] and sparse models such as SPLADE [51]. In addition, we aim to study the capability of GCP to affect unseen queries, as well as to drop query specificity and craft effective adversarial samples for groups of queries at a time.

6. AI Usage Disclosure

LLM-based tools were utilized to assist in the preparation of this work, specifically for polishing and improving the writing of the manuscript. All content, results, and conclusions remain the sole responsibility of the authors. No AI tool was used to generate research findings, tables, or references.

References

[1] J. Lin, R. Nogueira, A. Yates, Pretrained Transformers for Text Ranking: BERT and Beyond, Synthesis Lectures on Human Language Technologies, Springer International Publishing, Cham, 2022. URL: https://link.springer.com/10.1007/978-3-031-02181-7. doi:10.1007/978-3-031-02181-7.

[2] S. Bruch, C. Lucchese, F. M. Nardini, Efficient and effective tree-based and neural learning to rank, Foundations and Trends® in Information Retrieval 17 (2023) 1–123. URL: http://dx.doi.org/10.1561/1500000071. doi:10.1561/1500000071.

[3] J. Guo, Y. Fan, L. Pang, L. Yang, Q. Ai, H. Zamani, C. Wu, W. B. Croft, X. Cheng, A Deep Look into neural ranking models for information retrieval, Information Processing & Management 57 (2020) 102067. URL: https://www.sciencedirect.com/science/article/pii/S0306457319302390. doi:10.1016/j.ipm.2019.102067.

[4] Y. Fan, X. Xie, Y. Cai, J. Chen, X. Ma, X. Li, R. Zhang, J. Guo, Pre-training Methods in Information Retrieval, Foundations and Trends® in Information Retrieval 16 (2022) 178–317. URL: https://www.nowpublishers.com/article/Details/INR-100. doi:10.1561/1500000100, publisher: Now Publishers, Inc.

[5] Y. Wang, L. Lyu, A. Anand, Bert rankers are brittle: A study using adversarial document perturbations, in: Proceedings of the 2022 ACM SIGIR International Conference on Theory of Information Retrieval, ICTIR '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 115–120. URL: https://doi.org/10.1145/3539813.3545122. doi:10.1145/3539813.3545122.

[6] N. Reimers, I. Gurevych, Sentence-BERT: Sentence embeddings using Siamese BERT-networks, in: K. Inui, J. Jiang, V. Ng, X. Wan (Eds.), Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Association for Computational Linguistics, Hong Kong, China, 2019, pp. 3982–3992. URL: https://aclanthology.org/D19-1410/. doi:10.18653/v1/D19-1410.

[7] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with GPUs, IEEE Transactions on Big Data 7 (2019) 535–547.

[8] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, IEEE Trans. Pattern Anal. Mach. Intell. 42 (2020) 824–836. URL: https://doi.org/10.1109/TPAMI.2018.2889473. doi:10.1109/TPAMI.2018.2889473.

[9] Z. Zhong, Z. Huang, A. Wettig, D. Chen, Poisoning retrieval corpora by injecting adversarial passages, in: H. Bouamor, J. Pino, K. Bali (Eds.), Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Singapore, 2023, pp. 13764–13775. URL: https://aclanthology.org/2023.emnlp-main.849/. doi:10.18653/v1/2023.emnlp-main.849.

[10] C. Wu, R. Zhang, J. Guo, M. De Rijke, Y. Fan, X. Cheng, Prada: Practical black-box adversarial attacks against neural ranking models, ACM Trans. Inf. Syst. 41 (2023). URL: https://doi.org/10.1145/3576923. doi:10.1145/3576923.

[11] Y. Li, P. Eustratiadis, S. Lupart, E. Kanoulas, Unsupervised corpus poisoning attacks in continuous space for dense retrieval, in: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '25, Association for Computing Machinery, New York, NY, USA, 2025, p. 2452–2462. URL: https://doi.org/10.1145/3726302.3730110. doi:10.1145/3726302.3730110.

[12] A. Parry, M. Fröbe, S. MacAvaney, M. Potthast, M. Hagen, Analyzing adversarial attacks on sequence-to-sequence relevance models, in: Advances in Information Retrieval: 46th European

Conference on Information Retrieval, ECIR 2024, Glasgow, UK, March 24–28, 2024, Proceedings, Part II, Springer-Verlag, Berlin, Heidelberg, 2024, p. 286–302. URL: https://doi.org/10.1007/978-3-031-56060-6_19. doi:10.1007/978-3-031-56060-6_19.

[13] J. Liu, Y. Kang, D. Tang, K. Song, C. Sun, X. Wang, W. Lu, X. Liu, Order-disorder: Imitation adversarial attacks for black-box neural ranking models, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 2025–2039. URL: https://doi.org/10.1145/3548606.3560683. doi:10.1145/3548606.3560683.

[14] J. Song, J. Zhang, J. Zhu, M. Tang, Y. Yang, TRAttack: Text rewriting attack against text retrieval, in: S. Gella, H. He, B. P. Majumder, B. Can, E. Giunchiglia, S. Cahyawijaya, S. Min, M. Mozes, X. L. Li, I. Augenstein, A. Rogers, K. Cho, E. Grefenstette, L. Rimell, C. Dyer (Eds.), Proceedings of the 7th Workshop on Representation Learning for NLP, Association for Computational Linguistics, Dublin, Ireland, 2022, pp. 191–203. URL: https://aclanthology.org/2022.repl4nlp-1.20/. doi:10.18653/v1/2022.repl4nlp-1.20.

[15] N. Raval, M. Verma, One word at a time: adversarial attacks on retrieval models, 2020. URL: https://arxiv.org/abs/2008.02197. arXiv:2008.02197.

[16] Y.-A. Liu, R. Zhang, J. Guo, M. de Rijke, Y. Fan, X. Cheng, Multi-granular adversarial attacks against black-box neural ranking models, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 1391–1400. URL: https://doi.org/10.1145/3626772.3657704. doi:10.1145/3626772.3657704.

[17] A. Chertkov, O. Tsymboi, M. Pautov, I. Oseledets, Translate your gibberish: black-box adversarial attack on machine translation systems, Journal of Mathematical Sciences 285 (2024) 221–233.

[18] Y.-A. Liu, R. Zhang, J. Guo, M. de Rijke, W. Chen, Y. Fan, X. Cheng, Black-box adversarial attacks against dense retrieval models: A multi-view contrastive learning method, in: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, 2023, pp. 1647–1656.

[19] Z. Gyöngyi, H. Garcia-Molina, Web spam taxonomy, in: Proceedings of the First International Workshop on Adversarial Information Retrieval on the Web (AIRWeb), ACM, 2005, pp. 1–10.

[20] Y.-A. Liu, R. Zhang, J. Guo, M. de Rijke, Y. Fan, X. Cheng, Robust neural information retrieval: An adversarial and out-of-distribution perspective, ACM Trans. Inf. Syst. 44 (2025). URL: https://doi.org/10.1145/3768153. doi:10.1145/3768153.

[21] J. Liu, Y. Kang, D. Tang, K. Song, C. Sun, X. Wang, W. Lu, X. Liu, Order-disorder: Imitation adversarial attacks for black-box neural ranking models, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 2025–2039. URL: https://doi.org/10.1145/3548606.3560683. doi:10.1145/3548606.3560683.

[22] C. Castillo, B. D. Davison, Adversarial web search, Found. Trends Inf. Retr. 4 (2011) 377–486. URL: https://doi.org/10.1561/1500000021. doi:10.1561/1500000021.

[23] A. Bigdeli, N. Arabzadeh, E. Bagheri, C. L. A. Clarke, Empra: Embedding perturbation rank attack against neural ranking models, ACM Trans. Inf. Syst. 44 (2026). URL: https://doi.org/10.1145/3801943. doi:10.1145/3801943.

[24] Y.-A. Liu, R. Zhang, J. Guo, M. de Rijke, W. Chen, Y. Fan, X. Cheng, Topic-oriented adversarial attacks against black-box neural ranking models, in: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '23, Association for Computing Machinery, New York, NY, USA, 2023, p. 1700–1709. URL: https://doi.org/10.1145/3539618.3591777. doi:10.1145/3539618.3591777.

[25] J. Ebrahimi, A. Rao, D. Lowd, D. Dou, HotFlip: White-box adversarial examples for text classification, in: I. Gurevych, Y. Miyao (Eds.), Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), Association for Computational Linguistics, Melbourne, Australia, 2018, pp. 31–36. URL: https://aclanthology.org/P18-2006/. doi:10.18653/v1/P18-2006.

[26] J. Song, C. Jin, W. Zhao, A. McCallum, J.-Y. Lee, Comparing neighbors together makes it easy: Jointly comparing multiple candidates for efficient and effective retrieval, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, pp. 22255–22269.

[27] H. Zhang, Y. Gong, Y. Shen, J. Lv, N. Duan, W. Chen, Adversarial retriever-ranker for dense text retrieval, 2022. URL: https://arxiv.org/abs/2110.03611. arXiv:2110.03611.

[28] H. Zhuang, Z. Qin, R. Jagerman, K. Hui, J. Ma, J. Lu, J. Ni, X. Wang, M. Bendersky, RankT5: Fine-Tuning T5 for Text Ranking with Ranking Losses, in: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 2308–2313. URL: https://dl.acm.org/doi/10.1145/3539618.3592047. doi:10.1145/3539618.3592047.

[29] F. Jiang, Q. Xu, T. Drummond, T. Cohn, Boot and switch: Alternating distillation for zero-shot dense retrieval, in: H. Bouamor, J. Pino, K. Bali (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2023, Association for Computational Linguistics, Singapore, 2023, pp. 912–931. URL: https://aclanthology.org/2023.findings-emnlp.65/. doi:10.18653/v1/2023.findings-emnlp.65.

[30] Y. Chen, H. Gao, G. Cui, F. Qi, L. Huang, Z. Liu, M. Sun, Why should adversarial perturbations be imperceptible? rethink the research paradigm in adversarial NLP, in: Y. Goldberg, Z. Kozareva, Y. Zhang (Eds.), Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 2022, pp. 11222–11237. URL: https://aclanthology.org/2022.emnlp-main.771/. doi:10.18653/v1/2022.emnlp-main.771.

[31] Y. Lei, L. Ding, Y. Cao, C. Zan, A. Yates, D. Tao, Unsupervised dense retrieval with relevance-aware contrastive pre-training, in: A. Rogers, J. Boyd-Graber, N. Okazaki (Eds.), Findings of the Association for Computational Linguistics: ACL 2023, Association for Computational Linguistics, Toronto, Canada, 2023, pp. 10932–10940. URL: https://aclanthology.org/2023.findings-acl.695/. doi:10.18653/v1/2023.findings-acl.695.

[32] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense passage retrieval for open-domain question answering, in: B. Webber, T. Cohn, Y. He, Y. Liu (Eds.), Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, Online, 2020, pp. 6769–6781. URL: https://aclanthology.org/2020.emnlp-main.550/. doi:10.18653/v1/2020.emnlp-main.550.

[33] L. Xiong, C. Xiong, Y. Li, K. Tang, J. Liu, P. N. Bennett, J. Ahmed, A. Overwijk, Approximate nearest neighbor negative contrastive learning for dense text retrieval, CoRR abs/2007.00808 (2020). URL: https://arxiv.org/abs/2007.00808. arXiv:2007.00808.

[34] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, L. Deng, MS MARCO: A human generated machine reading comprehension dataset, in: T. R. Besold, A. Bordes, A. S. d'Avila Garcez, G. Wayne (Eds.), Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2016 co-located with the 30th Annual Conference on Neural Information Processing Systems (NIPS 2016), Barcelona, Spain, December 9, 2016, CEUR Workshop Proceedings, CEUR-WS.org, 2016. URL: https://ceur-ws.org/Vol-1773/CoCoNIPS_2016_paper9.pdf.

[35] N. Craswell, B. Mitra, E. Yilmaz, D. F. Campos, E. M. Voorhees, Overview of the trec 2019 deep learning track, ArXiv abs/2003.07820 (2020). URL: https://api.semanticscholar.org/CorpusID:253234683.

[36] N. Craswell, B. Mitra, E. Yilmaz, D. Campos, Overview of the TREC 2020 deep learning track, CoRR abs/2102.07662 (2021). URL: https://arxiv.org/abs/2102.07662. arXiv:2102.07662.

[37] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, in: J. Burstein, C. Doran, T. Solorio (Eds.), Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), Association for Computational Linguistics, 2019, pp. 4171–4186. URL: https://doi.org/10.18653/v1/n19-1423. doi:10.18653/V1/N19-1423.

[38] B. Warner, A. Chaffin, B. Clavié, O. Weller, O. Hallström, S. Taghadouini, A. Gallagher, R. Biswas,

F. Ladhak, T. Aarsen, N. Cooper, G. Adams, J. Howard, I. Poli, Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference, 2024. URL: https://arxiv.org/abs/2412.13663. arXiv:2412.13663.

[39] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, J. Mach. Learn. Res. 21 (2020).

[40] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, A. Rush, Transformers: State-of-the-art natural language processing, in: Q. Liu, D. Schlangen (Eds.), Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, Online, 2020, pp. 38–45. URL: https://aclanthology.org/2020.emnlp-demos.6/. doi:10.18653/v1/2020.emnlp-demos.6.

[41] R. Nogueira, Z. Jiang, R. Pradeep, J. Lin, Document ranking with a pretrained sequence-to-sequence model, in: T. Cohn, Y. He, Y. Liu (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2020, Association for Computational Linguistics, Online, 2020, pp. 708–718. URL: https://aclanthology.org/2020.findings-emnlp.63/. doi:10.18653/v1/2020.findings-emnlp.63.

[42] F. Busolin, C. Lucchese, F. M. Nardini, S. Orlando, R. Perego, S. Trani, A. Veneri, Efficient re-ranking with cross-encoders via early exit, in: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '25, Association for Computing Machinery, New York, NY, USA, 2025, p. 2534–2544. URL: https://doi.org/10.1145/3726302.3729962. doi:10.1145/3726302.3729962.

[43] J. Ni, G. Hernandez Abrego, N. Constant, J. Ma, K. Hall, D. Cer, Y. Yang, Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models, in: S. Muresan, P. Nakov, A. Villavicencio (Eds.), Findings of the Association for Computational Linguistics: ACL 2022, Association for Computational Linguistics, Dublin, Ireland, 2022, pp. 1864–1874. URL: https://aclanthology.org/2022.findings-acl.146/. doi:10.18653/v1/2022.findings-acl.146.

[44] H. Zhuang, Z. Qin, R. Jagerman, K. Hui, J. Ma, J. Lu, J. Ni, X. Wang, M. Bendersky, Rankt5: Fine-tuning t5 for text ranking with ranking losses, in: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '23, Association for Computing Machinery, New York, NY, USA, 2023, p. 2308–2313. URL: https://doi.org/10.1145/3539618.3592047. doi:10.1145/3539618.3592047.

[45] X. Song, A. Alexeev, P. Tan, C. Zhu, R. Futrell, H. Daumé III, Fast WordPiece tokenization, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021, pp. 2089–2103. URL: https://aclanthology.org.

[46] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, in: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2016, pp. 1715–1725. URL: https://www.aclweb.org/anthology/P16-1162.

[47] T. Kudo, J. Richardson, SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, Brussels, Belgium, 2018, pp. 66–71. URL: https://aclanthology.org/D18-2012.

[48] F. Ullah, M. Edwards, R. Ramdhany, R. Chitchyan, M. A. Babar, A. Rashid, Data exfiltration: A review of external attack vectors and countermeasures, Journal of Network and Computer Applications 101 (2018) 18–54.

[49] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM workshop on artificial intelligence and security, 2023, pp. 79–90.

[50] O. Khattab, M. Zaharia, Colbert: Efficient and effective passage search via contextualized late interaction over bert, in: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '20, Association for Computing Machinery, New York, NY, USA, 2020, p. 39–48. URL: https://doi.org/10.1145/3397271.3401075. doi:10.1145/

3397271.3401075.

[51] T. Formal, B. Piwowarski, S. Clinchant, SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking, Association for Computing Machinery, New York, NY, USA, 2021, p. 2288–2292. URL: https://doi.org/10.1145/3404835.3463098.